

ساخت دانلودر یوتیوب با پایتون

دانلود تک‌ویدیو و پلی‌لیست کامل با yt-dlp

یک پروژه عملی و گام‌به‌گام پایتون

از یک پوشه خالی شروع کنید و با برنامه‌ای کوچک تمام کنید که یک ویدیو، یا یک پلی‌لیست کامل را در پوشه‌ای به نام همان پلی‌لیست و با عنوان دقیق ویدیوها ذخیره می‌کند.

Alireza Khajehvandi

 LinkedIn •  GitHub • وبسایت 
 Instagram •  Telegram •  YouTube

روش استفاده از این آموزش

این راهنما کاملاً عملی و برای تازه‌کارهای پایتون نوشته شده است. فصل‌ها را به‌ترتیب انجام دهید و در هر **ایستگاه بررسی** توقف کنید. دستورها آماده‌ی کپی هستند؛ با این‌حال توضیح کوتاه کنار هر دستور را بخوانید تا بدانید چه کاری انجام می‌دهد.

- فصل‌های ۱ و ۲ هدف پروژه را توضیح می‌دهند و رایانه شما را آماده می‌کنند.
- فصل‌های ۳ و ۴ همه ابزارهای پایتون و همه تنظیم‌های yt-dlp را که در برنامه به کار رفته‌اند، هر کدام با یک مثال کوچک و قابل‌اجرا آموزش می‌دهند.
- فصل ۵ فایل downloader.py را گام‌به‌گام می‌سازد تا کامل شود.
- فصل ۶ برنامه را برای ویدیو، پلی‌لیست و صدا اجرا و آزمون می‌کند.
- فصل ۷ نشان می‌دهد استفاده واقعی چه مشکلاتی را آشکار کرد، علت آن‌ها چگونه پیدا شد و چرا هر اصلاح درست کار می‌کند.
- فصل ۸ مشکل‌های رایج را برطرف می‌کند و ایده‌هایی برای تمرین پیشنهاد می‌دهد.

نسخه‌های فارسی و انگلیسی، کد، دستورها، نتیجه‌های مورد انتظار و نکته‌های یکسانی دارند. فقط زبان و جهت صفحه‌های آن‌ها متفاوت است. پروژه کامل و فایل PDF هر دو نسخه در **GitHub** منتشر شده است.

نکته مهم

فقط ویدیوهایی را دانلود کنید که مالک آن‌ها هستید، مجوز انتشارشان اجازه دانلود می‌دهد یا برای ذخیره‌شان اجازه گرفته‌اید. قوانین استفاده از یوتیوب و حق نشر سازندگان را رعایت کنید. این آموزش برای یادگیری پایتون است و مسئولیت نحوه استفاده از برنامه با خود شماست.

فهرست مطالب

۱	۱	چه چیزی می‌سازیم
۱	۱.۱	رفتار برنامه
۱	۲.۱	برنامه چگونه کار می‌کند
۲	۳.۱	ساختار پروژه
۲	۴.۱	چه چیزهایی لازم دارید
۳	۲	آماده‌سازی پایتون و پروژه
۳	۱.۲	بررسی پایتون
۳	۲.۲	نصب FFmpeg
۴	۳.۲	تهیه پوشه پروژه
۴	۴.۲	ساخت محیط جداگانه
۵	۵.۲	نوشتن فایل نیازمندی‌ها
۵	۶.۲	نصب بسته‌ها
۶	۳	ابزارهای پایتونی که به کار می‌بریم
۶	۱.۳	ماژول‌ها و import
۶	۲.۳	متغیر، رشته و in
۷	۳.۳	تابع، return و مقدار پیش‌فرض
۷	۴.۳	تصمیم‌گیری با if و else
۷	۵.۳	لیست، تاپل و دیکشنری
۸	۶.۳	مسیرها با pathlib
۸	۷.۳	یافتن یک برنامه با shutil.which
۸	۸.۳	دستور with
۹	۹.۳	آرگومان‌های خط فرمان با argparse
۹	۱۰.۳	پرسیدن از کاربر با input
۹	۱۱.۳	مدیریت خطا با try و except
۱۰	۱۲.۳	جایگزینی متن با replace
۱۰	۱۳.۳	خط if __name__ == "__main__"

۱۱	۴ آشنایی با yt-dlp
۱۱	۱.۴ نگاه اول از داخل ترمینال
۱۱	۲.۴ به کار بردن yt-dlp در پایتون
۱۲	۳.۴ تنظیم‌هایی که برنامه ما به کار می‌برد
۱۲	۱.۳.۴ الگوی خروجی: نام‌گذاری فایل‌ها
۱۲	۲.۳.۴ انتخاب قالب: تعیین کیفیت
۱۳	۳.۳.۴ فقط صدا: پس‌پردازنده‌ها
۱۳	۴.۳.۴ کوکی‌ها: وقتی یوتیوب از شما می‌خواهد وارد شوید

۱۵	۵ نوشتن گام به گام برنامه
۱۵	۱.۵ گام ۱: توصیف فایل
۱۵	۲.۵ گام ۲: وارد کردن ابزارها و تعیین پوشه پیش‌فرض
۱۶	۳.۵ گام ۳: تشخیص پلی‌لیست
۱۶	۴.۵ گام ۴: انتخاب نام فایل‌ها
۱۶	۵.۵ گام ۵: انتخاب کیفیت
۱۷	۶.۵ گام ۶: جمع کردن همه تنظیم‌ها
۱۸	۷.۵ گام ۷: دانلود
۱۸	۸.۵ گام ۸: خواندن خط فرمان
۱۹	۹.۵ گام ۹: شروع برنامه
۲۰	۱۰.۵ برنامه کامل

۲۳	۶ اجرا و آزمون برنامه
۲۳	۱.۶ نمایش صفحه راهنما
۲۳	۲.۶ دانلود یک ویدیو
۲۴	۳.۶ دانلود یک پلی‌لیست کامل
۲۴	۴.۶ ذخیره فقط صدا
۲۵	۵.۶ انتخاب پوشه ذخیره
۲۵	۶.۶ روش‌های دیگر اجرا
۲۵	۷.۶ توقف و ادامه در زمانی دیگر
۲۶	۸.۶ آزمون‌های آفلاین

۲۸	۷ درس‌هایی از استفاده واقعی
۲۸	۱.۷ روشی ساده برای هر باگ
۲۸	۲.۷ درس ۱: قطع شبکه نیمی از ویدیو را جا گذاشت
۲۸	۱.۲.۷ نشانه
۲۹	۲.۲.۷ خواندن شواهد

۲۹	۳.۲.۷ پیدا کردن علت
۲۹	۴.۲.۷ اصلاح و آزمون
۳۰	۳.۷ درس ۲: ترمینال پیوند را تغییر داد
۳۰	۱.۳.۷ نشانه
۳۰	۲.۳.۷ چرا مهم است
۳۰	۳.۳.۷ اصلاح
۳۰	۴.۷ درس ۳: دانلودها را از کد و از گیت جدا نگه دارید
۳۱	۵.۷ درس ۴: یک قابلیت تازه، فقط صدا
۳۱	۶.۷ درس ۵: توقف مؤدبانه
۳۱	۷.۷ خلاصه تغییرها

۸ رفع اشکال و گام‌های بعدی ۳۳

۳۳	۱.۸ مشکل‌های رایج
۳۴	۲.۸ تمیز نگه داشتن پروژه در گیت
۳۵	۳.۸ ایده‌هایی برای تمرین

فصل ۱

چه چیزی می‌سازیم

هدف فصل

بفهمید برنامه‌نمایی چه کاری انجام می‌دهد، در یک نگاه چگونه کار می‌کند و پروژه چه فایل‌هایی دارد.

یک برنامه کوتاه پایتون به نام `downloader.py` می‌نویسیم. پیوند یک ویدیوی یوتیوب را در ترمینال به آن می‌دهید و برنامه ویدیو را روی رایانه شما ذخیره می‌کند.

۱.۱ رفتار برنامه

- **پیوند یک ویدیو:** برنامه همان ویدیو را دانلود می‌کند و نام فایل را دقیقاً عنوان آن در یوتیوب می‌گذارد؛ مثلاً `Me at the zoo.mp4`.
- **پیوند یک پلی‌لیست:** برنامه پوشه‌ای به نام پلی‌لیست می‌سازد و همه ویدیوها را در آن دانلود می‌کند. هر فایل عنوان دقیق خودش را نگه می‌دارد و یک شماره هم می‌گیرد تا ترتیب پلی‌لیست حفظ شود: `01 - First video.mp4`، `02 - Second video.mp4` و به همین ترتیب.
- **پوشه ذخیره:** با گزینه `-o FOLDER` محل ذخیره را انتخاب می‌کنید. اگر آن را ننویسید، فایل‌ها در پوشه `downloads` کنار `downloader.py` ذخیره می‌شوند؛ فرقی نمی‌کند برنامه را از کجا اجرا کرده باشید. گیت این پوشه را نادیده می‌گیرد؛ پس ویدیوها هرگز وارد مخزن شما نمی‌شوند.
- **فقط صدا:** با گزینه `--audio` برنامه فقط صدا را به شکل فایل `mp3` ذخیره می‌کند؛ مناسب برای موسیقی، پادکست و درس‌های زبان.
- **بهترین کیفیت:** بهترین تصویر و بهترین صدا گرفته و در یک فایل `mp4` ترکیب می‌شوند که روی گوشی، تلویزیون و همه پخش‌کننده‌های رایج پخش می‌شود.
- **رفتار دوستانه و مقاوم:** اگر پیوند را فراموش کنید، برنامه آن را از شما می‌پرسد. اگر یکی از ویدیوهای پلی‌لیست خصوصی یا حذف شده باشد، از آن می‌گذرد و بقیه را ادامه می‌دهد. پس از قطع شبکه خودکار دوباره تلاش می‌کند و با `Ctrl+C` مؤدبانه متوقف می‌شود تا همان دستور بعداً ادامه دهد.

۲.۱ برنامه چگونه کار می‌کند

کار سخت، یعنی گفت‌وگو با یوتیوب، انتخاب قالب‌ها، دانلود تکه‌تکه و نمایش پیشرفت، را `yt-dlp` انجام می‌دهد؛ یک کتابخانه متن‌باز و پرطرفدار پایتون. کار ما **آماده‌کردن تنظیم‌های درست** و سپردن پیوند به آن است.

پیوند شما در ترمینال

↓ argparse آن را می‌خواند

پلی‌لیست است؟ چه نامی؟ چه کیفیتی؟ کدام پوشه؟

↓ یک دیکشنری پایتون از تنظیم‌ها

yt-dlp دانلود می‌کند

↓ FFmpeg تصویر و صدا را ترکیب می‌کند

برای هر ویدیو یک فایل کامل .mp4

۳.۱ ساختار پروژه

در پایان، پوشه شما به این شکل است. برای اجرای برنامه فقط فایل‌های پوشه code/ لازم‌اند؛ پوشه docs/ همین آموزش را نگه می‌دارد.

```
prj-youtube-downloader/
|-- code/
|   |-- downloader.py      # the complete program (119 lines)
|   |-- requirements.txt   # the packages to install
|   |-- test_downloader.py # nine offline tests
|   |-- README.md         # how to run the code
|   `-- downloads/        # your videos (created automatically, not in Git)
|-- docs/
|   |-- english/          # this book (English, XeLaTeX)
|   |-- persian/          # the same book in Persian (XePersian)
|   `-- README.md         # how to build the books
|-- .gitignore            # files Git must not save (videos, ...)
|-- LICENSE
`-- README.md             # project overview
```

۴.۱ چه چیزهایی لازم دارید

- رایانه‌ای با ویندوز، مک یا لینوکس و اتصال به اینترنت.
- پایتون نسخه ۳/۱۰ یا جدیدتر (روش بررسی در فصل ۲ آمده است).
- FFmpeg؛ ابزاری رایگان برای ترکیب تصویر و صدا (در فصل ۲ نصب می‌شود).
- یک ویرایشگر متن. Visual Studio Code گزینه رایگان و خوبی است.

به تجربه قبلی با yt-dlp نیاز ندارید. همه مفاهیم‌های پایتونی که در برنامه به کار رفته‌اند، در فصل ۳ با مثال توضیح داده شده‌اند.

ایستگاه بررسی

می‌توانید در یک جمله بگویید با پیوند یک ویدیو چه اتفاقی می‌افتد، با پیوند یک پلی‌لیست چه اتفاقی می‌افتد و اگر پوشه‌ای انتخاب نکنید، فایل‌ها کجا ذخیره می‌شوند.

فصل ۲

آماده سازی پایتون و پروژه

هدف فصل

پایتون را بررسی کنید، FFmpeg را نصب کنید، پوشه پروژه و یک محیط جداگانه پایتون بسازید و بسته ها را نصب کنید.

۱.۲ بررسی پایتون

یک ترمینال باز کنید: در مک و لینوکس برنامه Terminal، در ویندوز برنامه PowerShell یا ترمینال داخل ویرایشگر. سپس اجرا کنید:

```
python --version
```

در بعضی سیستم های مک یا لینوکس باید از دستور python3 استفاده کنید. هر نتیجه ای از Python 3.10 به بالا، مانند Python 3.12.11، مناسب است. اگر پایتون نصب نیست، آن را از <https://www.python.org/downloads/> نصب کنید. در ویندوز هنگام نصب، گزینه Add python.exe to PATH را علامت بزنید.

۲.۲ نصب FFmpeg

یوتیوب بهترین تصویر و بهترین صدا را به صورت دو فایل جدا نگه می دارد. FFmpeg آن ها را در یک فایل mp4 ترکیب می کند. آن را با مدیر بسته سیستم خود نصب کنید:

```
sudo apt install ffmpeg      # Ubuntu / Debian Linux
brew install ffmpeg          # macOS with Homebrew
winget install Gyan.FFMpeg    # Windows PowerShell
```

ترمینال را ببندید و دوباره باز کنید؛ سپس نصب را بررسی کنید:

```
ffmpeg -version
```

خط اول باید با ffmpeg version شروع شود. برنامه بدون FFmpeg هم کار می کند، اما در آن حالت فقط می تواند فایل هایی با کیفیت پایین تر را ذخیره کند که از قبل تصویر و صدا را با هم دارند (دلیلش در بخش ۲.۳.۴ آمده است).

۳.۲ تهیه پوشه پروژه

یکی از دو روش زیر را انتخاب کنید.

روش الف: همه چیز را خودتان بسازید (پیشنهادی برای یادگیری)

یک پوشه خالی برای پروژه و یک پوشه code در داخل آن بسازید و وارد آن شوید:

```
mkdir prj-youtube-downloader
cd prj-youtube-downloader
mkdir code
cd code
```

همه فایل‌ها را هنگام خواندن فصل‌های ۲، ۵ و ۶ در همین پوشه تایپ خواهید کرد.

روش ب: پروژه کامل را دریافت کنید

```
git clone https://github.com/alirezaaies/prj-youtube-downloader.git
cd prj-youtube-downloader/code
```

اگر به جای آن فایل فشرده را از گیت‌هاب دانلود کرده‌اید، آن را استخراج کنید و ترمینال را در پوشه code آن باز کنید.

۴.۲ ساخت محیط جداگانه

محیط مجازی جعبه‌ای خصوصی از بسته‌ها برای یک پروژه است. این محیط بسته‌های این پروژه را از پروژه‌های دیگر پایتون جدا نگه می‌دارد تا هرگز با هم تداخل نکنند. یکی از روش‌های زیر را به کار ببرید.

محیط داخلی پایتون (venv) در مک یا لینوکس:

```
python3 -m venv .venv
source .venv/bin/activate
```

محیط داخلی پایتون (venv) در پاورشل ویندوز:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

کوندا (Anaconda یا Miniconda) در هر سیستمی:

```
conda create -n ytdl python12.3= -y
conda activate ytdl
```

اکنون ترمینال پیش از نشانگر خود عبارت (.venv) یا (ytdl) را نشان می‌دهد. هر بار که ترمینال تازه‌ای برای این پروژه باز می‌کنید، محیط را دوباره فعال کنید.

نکته مهم

اگر پاورشل اجازه اجرای Activate.ps1 را نداد، یک بار دستور Set-ExecutionPolicy -Scope CurrentUser RemoteSigned را اجرا کنید، پاسخ Y بدهید و دوباره امتحان کنید.

۵.۲ نوشتن فایل نیازمندی‌ها

فایل requirements.txt بسته‌های لازم یک پروژه را فهرست می‌کند تا هر کسی بتواند همه آن‌ها را با یک دستور نصب کند. در روش الف، فایل requirements.txt را در پوشه code با محتوای زیر بسازید (در روش ب این فایل از قبل وجود دارد):

Listing ۱.۲: code/requirements.txt

```
# yt-dlp talks to YouTube; [default] adds its recommended helpers.
# YouTube changes often, so keep yt-dlp up to date (pip install -U ...).
yt-dlp[default]19.8.2026=<
# JavaScript runtime that yt-dlp needs to read YouTube's player.
deno0.2=<
```

- خط‌هایی که با # شروع می‌شوند، توضیح برای انسان هستند و pip آن‌ها را نادیده می‌گیرد.
- yt-dlp[default] بسته yt-dlp را همراه با ابزارهای کمکی پیشنهادی‌اش نصب می‌کند. بخش داخل کروشه افزونه (extra) نام دارد.
- 19.8.2026=> یعنی «این نسخه یا هر نسخه جدیدتر». نسخه‌های جدیدتر را عمداً مجاز گذاشته‌ایم؛ یوتیوب مرتب تغییر می‌کند و yt-dlp هم برای هماهنگی با آن به‌روز می‌شود.
- deno یک موتور کوچک جاوااسکریپت است. پخش‌کننده وب یوتیوب معماهایی به زبان جاوااسکریپت دارد و yt-dlp برای حل آن‌ها از deno کمک می‌گیرد. بدون آن، بعضی قالب‌ها در دسترس نیستند یا دانلود شکست می‌خورد.

۶.۲ نصب بسته‌ها

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

خط اول pip، یعنی نصب‌کننده بسته‌های پایتون، را به‌روز می‌کند. خط دوم فایل requirements.txt را می‌خواند (-r) و همه بسته‌های فهرست‌شده در آن را نصب می‌کند. نوشتن python -m pip به‌جای pip تضمین می‌کند که pip همان محیط فعال به کار برود.

به یک تفاوت کوچک در نام‌ها دقت کنید: بسته با نام yt-dlp (با خط تیره) نصب می‌شود، اما در پایتون با نام yt_dlp (با زیرخط) وارد می‌شود؛ چون نام‌ها در پایتون نمی‌توانند خط تیره داشته باشند.

ایستگاه بررسی

سه دستور زیر را اجرا کنید. هر کدام یک شماره نسخه چاپ می‌کند. اگر هر سه کار کردند، محیط شما آماده است.

```
python -c "import yt_dlp; print(yt_dlp.version.__version__)"
deno --version
ffmpeg -version
```

فصل ۳

ابزارهای پایتونی که به کار می‌بریم

هدف فصل

همه مفاهیم‌های پایتونی موجود در `download.py` را، هر کدام با یک مثال کوچک که خودتان می‌توانید اجرا کنید، یاد بگیرید.

همه مثال‌ها را می‌توانید در **پوسته تعاملی پایتون** امتحان کنید. در ترمینال فعال‌شده دستور `python` را بنویسید؛ نشانگر به `>>>` تغییر می‌کند. یک خط بنویسید و `Enter` را بزنید تا نتیجه‌اش را ببینید. با `exit()` از پوسته خارج شوید. در مثال‌ها، خط‌هایی که با `#` شروع می‌شوند توضیح هستند و خروجی مورد انتظار در توضیحی مانند `True -> #` نشان داده شده است.

۱.۳ ماژول‌ها و `import`

ماژول فایلی از کدهای آماده پایتون است. `import` آن را در برنامه شما در دسترس قرار می‌دهد. بعضی ماژول‌ها همراه خود پایتون هستند (**کتابخانه استاندارد**) و بعضی دیگر، مانند `yt_dlp`، با `pip` نصب می‌شوند.

```
import shutil # use it as shutil.which(...)
from pathlib import Path # take only Path from the pathlib module
from yt_dlp import YoutubeDL # take only YoutubeDL from yt_dlp
```

برنامه ما سه ماژول استاندارد را به کار می‌برد: `argparse` (خواندن خط فرمان)، `shutil` (یافتن برنامه‌ها) و `pathlib` (کار با پوشه‌ها). یک کتابخانه نصب‌شده هم دارد: `yt_dlp`.

۲.۳ متغیر، رشته و `in`

متغیر نامی برای یک مقدار است. مقدارهای متنی **رشته** نام دارند و داخل گیومه نوشته می‌شوند. عملگر `in` بررسی می‌کند که آیا یک رشته داخل رشته دیگری آمده است یا نه، و نتیجه `True` یا `False` می‌دهد (یک مقدار بولی).

```
url = "https://www.youtube.com/playlist?list=PL123"
print("list=" in url) # -> True
print("list=" in "watch?v=abc") # -> False
```

افاسترینگ (رشته‌ای که پیش از گیومه‌اش حرف `f` آمده است) مقدارها را با آکولاد داخل متن قرار می‌دهد:

```
folder = "/home/sara/Videos"
```

```
print(f"Saving to: {folder}") # -> Saving to: /home/sara/Videos
```

متد `strip()` فاصله‌ها و شکستن خط‌ها را از دو سر یک رشته حذف می‌کند. این کار برای متنی که کاربر جای‌گذاری می‌کند، مفید است:

```
print(" https://youtu.be/abc ".strip()) # -> https://youtu.be/abc
```

۳.۳ تابع، return و مقدار پیش‌فرض

تابع بلوکی نام‌دار از کد است که می‌توانید بارها آن را صدا بزنید. `def` آن را می‌سازد و `return` نتیجه را به فراخواننده برمی‌گرداند. رشته‌ی درست زیر خط `def` یک **داک‌استرینگ** است؛ توضیحی کوتاه برای انسان. خط‌هایی که تورفتگی (چهار فاصله) دارند، به تابع تعلق دارند.

```
def greet(name, greeting="Hello"):
    """Build a short greeting."""
    return f"{greeting}, {name}!"

print(greet("Sara")) # -> Hello, Sara!
print(greet("Sara", "Hi")) # -> Hi, Sara!
```

`greeting="Hello"` یک **مقدار پیش‌فرض** است: وقتی فراخواننده آن آرگومان را ندهد، همین مقدار به کار می‌رود. تابع `download` ما برای پوشه و مرورگر از همین ایده استفاده می‌کند. مقدار ویژه `None` یعنی «چیزی داده نشده است».

۴.۳ تصمیم‌گیری با `if` و `else`

`if` بلوک خود را فقط وقتی اجرا می‌کند که شرط درست باشد؛ در غیر این صورت `else` اجرا می‌شود. دستور `return` تابع را بی‌درنگ تمام می‌کند؛ پس کدهای بعد از آن اجرا نمی‌شوند.

```
def size_word(count):
    if count > 1:
        return "many videos"
    return "one video"

print(size_word(5)) # -> many videos
print(size_word(1)) # -> one video
```

پایتون مقدارهای خالی را هم نادرست در نظر می‌گیرد: `""`، `None` و `0` نادرست هستند و هر رشته‌ی غیرخالی درست است. پس `if browser:` یعنی «اگر نام مرورگری داده شده است». عملگر `or` نخستین مقدار درست را برمی‌گرداند:

```
print(None or "typed by user") # -> typed by user
print("from command" or "typed") # -> from command
```

۵.۳ لیست، تاپل و دیکشنری

- **لیست** مقدارها را به‌ترتیب نگه می‌دارد و قابل تغییر است: `["a", "b"]`.
- **تاپل** مانند لیست است، اما تغییر نمی‌کند: `("firefox",)`. تاپل یک‌عضوی به ویرگول پایانی نیاز دارد؛ `("firefox")` فقط یک رشته است.

• **دیکشنری جفت‌های کلید: مقدار** را نگه می‌دارد؛ مانند فرمی کوچک با خانه‌های برچسب‌دار.

```
options = {"format": "best", "ignoreerrors": True}
print(options["format"])          # -> best
options["cookiesfrombrowser"] = ("firefox",)  # add a new key
print(len(options))                # -> 3
```

yt-dlp همهٔ تنظیم‌هایش را به شکل یک دیکشنری دریافت می‌کند؛ پس دیکشنری مهم‌ترین نوع داده در برنامهٔ ماست.

۶.۳ مسیرها با pathlib

Path مسیر یک فایل یا پوشه را نشان می‌دهد و در ویندوز، مک و لینوکس یکسان کار می‌کند.

```
from pathlib import Path

p = Path("~/Videos").expanduser()  # ~ becomes your home folder
print(p)                            # -> /home/sara/Videos
print(p / "Music")                  # -> /home/sara/Videos/Music
print(Path("code/downloader.py").resolve().parent)  # full folder path
p.mkdir(parents=True, exist_ok=True)  # create it (and its parents) if missing
```

• `__file__` متغیری ویژه است که مسیر فایل پایتون در حال اجرا را نگه می‌دارد.

• `resolve()` مسیر را به مسیر کامل (مطلق) تبدیل می‌کند.

• `parent` پوشه‌ای است که آن مسیر را در بر دارد.

• `exist_ok=True` یعنی «اگر پوشه از قبل وجود دارد، خطا نده».

بنابراین `Path(__file__).resolve().parent` یعنی «پوشه‌ای که این اسکریپت در آن است». این دقیقاً پوشهٔ پیش‌فرض ذخیره در برنامهٔ ماست.

۷.۳ یافتن یک برنامه با shutil.which

`shutil.which(name)` یک دستور را روی رایانهٔ شما جست‌وجو می‌کند. اگر دستور وجود داشته باشد، مسیری که آن را برمی‌گرداند و اگر نباشد، `None`.

```
import shutil
print(shutil.which("ffmpeg"))      # -> /usr/bin/ffmpeg
print(shutil.which("no-such-tool")) # -> None
```

۸.۳ دستور with

بعضی چیزها باید باز و بعداً به‌درستی بسته شوند. `with` هر دو کار را انجام می‌دهد: آن را باز می‌کند، بلوک توراته را اجرا می‌کند و در پایان می‌بندد؛ حتی اگر خطایی رخ دهد.

```
with open("note.txt", "w") as f:  # open a file for writing
    f.write("Hello")
# here the file is already closed
```

ما: `with YoutubeDL(...) as ydl:` را به همین شکل به کار می‌بریم.

۹.۳ آرگومان‌های خط فرمان با `argparse`

`argparse` کلمه‌هایی را می‌خواند که پس از `python script.py` نوشته می‌شوند. این ماژول به‌طور خودکار صفحه راهنمای `--help` می‌سازد و خطاهای روشن نشان می‌دهد. این کد را با نام `hello.py` ذخیره کنید:

```
import argparse

parser = argparse.ArgumentParser(description="Say hello.")
parser.add_argument("name", nargs="?", help="who to greet")
parser.add_argument("-t", "--times", default="1", help="how many times")
args = parser.parse_args()
print(f"Hello {args.name}! " * int(args.times))
```

```
python hello.py Sara          # -> Hello Sara!
python hello.py Sara -t 2     # -> Hello Sara! Hello Sara!
python hello.py --help        # shows the automatic help page
```

- "name" خط تیره ندارد؛ پس آرگومانی **موقعیتی** است و مقدارش بر اساس جایگاهش داده می‌شود.
- `nargs="?"` آن را اختیاری می‌کند؛ اگر داده نشود، مقدارش `None` است.
- `-t`، `--times` یک **گزینه** است: نامی کوتاه و نامی بلند برای یک تنظیم، همراه با مقدار پیش‌فرض.
- پس از `parse_args()` مقدارها با `args.name` و `args.times` در دسترس هستند.

بعضی گزینه‌ها کلید ساده‌ای برای روشن یا خاموش کردن هستند و مقداری نمی‌گیرند. این گزینه‌ها با تنظیم `action="store_true"` ساخته می‌شوند. در این حالت اگر گزینه نوشته شود مقدارش `True` و اگر نوشته نشود `False` است.

```
parser.add_argument("-l", "--loud", action="store_true", help="shout")
# python hello.py Sara          -> args.loud is False
# python hello.py Sara --loud   -> args.loud is True
```

۱۰.۳ پرسیدن از کاربر با `input`

`input(message)` پیامی نشان می‌دهد، منتظر می‌ماند تا کاربر `Enter` را بزند و متن تایپ‌شده را به شکل رشته برمی‌گرداند:

```
link = input("Paste a link: ").strip()
print(f"You pasted {link}")
```

۱۱.۳ مدیریت خطا با `try` و `except`

وقتی هنگام اجرای برنامه مشکلی پیش بیاید، پایتون یک **استثنا** (`exception`) ایجاد می‌کند و اگر کسی آن را مدیریت نکند، با یک گزارش خطای طولانی (`traceback`) متوقف می‌شود. `try` یک بلوک را اجرا می‌کند؛ اگر استثنای نام‌برده‌شده در آن رخ دهد، به جای توقف برنامه، بلوک `except` متناظر اجرا می‌شود.

```
try:
    number = int(input("Type a number: "))
except ValueError:
    print("That was not a number.")
```

زدن Ctrl+C در ترمینال، استثنای KeyboardInterrupt را ایجاد می‌کند. برنامه ما آن را می‌گیرد تا به جای گزارش خطا، یک جمله دوستانه چاپ کند. فقط استثناهایی را بگیرید که انتظارشان را دارید؛ گرفتن همه خطاها، باگ‌های واقعی را هم پنهان می‌کند.

۱۲.۳ جایگزینی متن با replace

text.replace(old, new) یک رونوشت از رشته برمی‌گرداند که در آن هر old به new تبدیل شده است. جایگزینی با رشته خالی یعنی حذف. در کد پایتون یک بک‌اسلش به شکل "\\" نوشته می‌شود، چون یک بک‌اسلش تنها یک دنباله فرار مانند \n (خط جدید) را شروع می‌کند.

```
link = "https://youtu.be/abc\\?si\\=xyz"
print(link) # -> https://youtu.be/abc\?si\=xyz
print(link.replace("\\", "")) # -> https://youtu.be/abc?si=xyz
```

۱۳.۳ خط if __name__ == "__main__"

وقتی یک فایل را مستقیم اجرا می‌کنید، پایتون متغیر __name__ آن را برابر "__main__" می‌گذارد. وقتی فایل دیگری (مثلاً فایل آزمون) آن را وارد می‌کند، مقدار __name__ نام ماژول است. پس این بلوک یعنی «برنامه را فقط وقتی شروع کن که این فایل مستقیم اجرا شده باشد»:

```
if __name__ == "__main__":
    main()
```

به لطف آن، آزمون‌ها می‌توانند تابع‌های ما را وارد کنند بدون اینکه داندلودی شروع شود.

خودتان امتحان کنید

پوسته پایتون را باز کنید و مثال‌های pathlib و shutil.which را روی رایانه خودتان اجرا کنید. نتیجه‌ها را با توضیحات مقایسه کنید.

ایستگاه بررسی

می‌توانید تابع، دیکشنری، Path(__file__).resolve().parent، shutil.which، argparse و try/except را هر کدام در یک جمله توضیح دهید.

فصل ۴

آشنایی با yt-dlp

هدف فصل

بفهمید yt-dlp چگونه از داخل پایتون به کار می‌رود و هر تنظیم در دیکشنری تنظیم‌های ما چه کاری انجام می‌دهد.

yt-dlp یک دانلودر رایگان و متن‌باز برای یوتیوب و بسیاری از سایت‌های دیگر است (<https://github.com/yt-dlp/yt-dlp>). آن را به دو شکل می‌توان به کار برد: به صورت دستور ترمینال و به صورت کتابخانه پایتون. برنامه ما از کتابخانه استفاده می‌کند، اما امتحان کردن دستور ترمینال در ابتدا کمک می‌کند ببینید چه کاری انجام می‌دهد.

۱.۴ نگاه اول از داخل ترمینال

```
yt-dlp --version
yt-dlp -F "https://www.youtube.com/watch?v=jNQXAC9IVRw"
```

دستور دوم چیزی دانلود نمی‌کند. گزینه F - همه قالب‌هایی را که یوتیوب برای آن ویدیو ارائه می‌کند، فهرست می‌کند. ردیف‌هایی فقط صدا، ردیف‌هایی فقط تصویر و چند ردیف با هر دو را خواهید دید. دقت کنید که بهترین کیفیت‌ها همیشه فقط تصویر یا فقط صدا هستند. به همین دلیل برای ترکیب آن‌ها به FFmpeg نیاز داریم.

نکته مهم

پیوند یوتیوب را در ترمینال همیشه داخل گیومه بنویسید. پیوندها اغلب نویسه & دارند که ترمینال در غیر این صورت آن را جداکننده دستورها در نظر می‌گیرد.

۲.۴ به کار بردن yt-dlp در پایتون

در پایتون، yt-dlp فقط به سه گام نیاز دارد: یک دیکشنری تنظیم بنویسید، با آن یک شیء YoutubeDL بسازید و download را با یک لیست از پیوندها صدا بزنید.

```
from yt_dlp import YoutubeDL

options = {"outtmpl": "%(title)s.%(ext)s"}
with YoutubeDL(options) as ydl:
```

```
result = ydl.download(["https://www.youtube.com/watch?v=jNQXAC9IVRw"])
print(result) # -> 0 means success
```

download وقتی همه پیوندها موفق باشند، 0 و در صورت بروز مشکل عددی دیگر برمی گرداند. برنامه ما از همین عدد برای چاپ پیام پایانی استفاده می کند.

۳.۴ تنظیم هایی که برنامه ما به کار می برد

کلید	کاربرد
format	کدام تصویر و صدا دانلود شود.
format_sort	وقتی چند قالب «بهترین» هستند، کدام ترجیح داده شود.
merge_output_format	قالب فایل هنگام ترکیب تصویر و صدا: mp4.
paths	پوشه پایه دانلودها: {"home": folder}.
outtmpl	الگوی خروجی ؛ یعنی نام گذاری هر فایل.
ignoreerrors	True: از ویدیوی ناموفق بگذر و بقیه را ادامه بده.
retries	پس از قطع شبکه، تا ۱۰ بار دوباره تلاش کن.
postprocessors	کارهایی که پس از دانلود انجام می شوند، مانند تبدیل به MP3.
cookiesfrombrowser	استفاده از ورود یوتیوب یک مرورگر: ("firefox",).

۱.۳.۴ الگوی خروجی: نام گذاری فایل ها

الگوی خروجی یک رشته معمولی است که خانه های ویژه ای به شکل s(field)% دارد. yt-dlp هر خانه را با اطلاعات ویدیو جایگزین می کند:

خانه	جایگزین آن
s(title)%	عنوان دقیق ویدیو
s(ext)%	پسوند فایل، مانند mp4
s(playlist_title)%	عنوان پلی لیست
s(playlist_index)02d	جایگاه در پلی لیست، به صورت دو رقمی

s یعنی «به شکل متن» و 02d یعنی «به شکل عدد صحیح با دست کم دو رقم و پرشده با صفر»؛ پس ۳ به 03 تبدیل می شود. این صفرها باعث می شوند فایل ها در مدیر فایل به درستی مرتب شوند (02 پیش از 10).

نویسه / در الگو یک پوشه می سازد. پس برای پلی لیستی به نام Python Basics که ویدیوی سومش Loops نام دارد، الگوی

```
%(playlist_title)s/%(playlist_index)02d - %(title)s.%(ext)s
```

نتیجه Python Basics/03 - Loops.mp4 را می سازد. اگر عنوانی نویسه هایی داشته باشد که در نام فایل مجاز نیستند (مانند / یا : در ویندوز)، yt-dlp خودش آن ها را به شکلی امن جایگزین می کند. عنوان های فارسی و دیگر زبان ها همان طور که هستند، حفظ می شوند.

۲.۳.۴ انتخاب قالب: تعیین کیفیت

مقدار format یک عبارت کوچک است:

```
bestvideo*+bestaudio/best
```

• `bestvideo*` بهترین جریان تصویر است؛ ستاره اجازه می‌دهد جریان‌هایی هم که اتفاقاً صدا دارند، انتخاب شوند.

• `bestaudio` + بهترین جریان صدا را اضافه می‌کند؛ این دو با `FFmpeg` ترکیب می‌شوند.

• `best` / گزینه جایگزین است: «اگر ممکن نبود، بهترین فایل تکی را بگیر که از قبل تصویر و صدا را با هم دارد».

ترکیب کردن به `FFmpeg` نیاز دارد. بدون آن فقط `best` را درخواست می‌کنیم که هرگز به ترکیب نیاز ندارد. بهترین فایل‌های تکی یوتیوب معمولاً فقط کیفیت `360p` دارند؛ پس نصب `FFmpeg` به شدت توصیه می‌شود. `format_sort` تعیین می‌کند از میان چند قالب خوب کدام برنده شود. یوتیوب اغلب کدک‌های جدیدی (`Opus`، `VP9`، `AV1`) ارائه می‌کند که بعضی تلویزیون‌ها، گوشی‌ها و پخش‌کننده‌های قدیمی نمی‌توانند بازشان کنند. لیست

```
["vcodec:h264", "res", "acodec:m4a"]
```

یعنی: اول تصویر `H.264`، سپس بیشترین وضوح و سپس صدای `AAC (m4a)` را ترجیح بده. نتیجه یک فایل `mp4` است که همه‌جا پخش می‌شود و معمولاً کیفیت `Full HD (1080p)` دارد.

۳.۳.۴ فقط صدا: پس‌پردازنده‌ها

برای فقط صدا، قالب به‌سادگی `bestaudio/best` است: بهترین جریان صدا، یا اگر صدای جداگانه‌ای نبود، بهترین فایل. بهترین صدای یوتیوب یک فایل `m4a` (`AAC`) یا `webm` (`Opus`) است. برای گرفتن فایل `mp3`، که همه گوشی‌ها، خودروها و پخش‌کننده‌های قدیمی آن را می‌شناسند، یک **پس‌پردازنده** درخواست می‌کنیم؛ گامی که `yt-dlp` پس از دانلود اجرا می‌کند.

```
"postprocessors": [
    {"key": "FFmpegExtractAudio", "preferredcodec": "mp3",
     "preferredquality": "192"},
]
```

• `postprocessors` یک **لیست** است، چون چند گام می‌توانند پشت سر هم اجرا شوند؛ هر گام یک دیکشنری است.

• `"key"` نام گام است. `FFmpegExtractAudio` از `FFmpeg` می‌خواهد فایل دانلودشده را به صدا تبدیل کند.

• `preferredcodec` قالب خروجی و `preferredquality` نرخ بیت را بر حسب کیلوبیت بر ثانیه تعیین می‌کند: `192` برای گفتار واضح و موسیقی خوب با حجم متوسط مناسب است (حدود `۱/۴` مگابایت برای هر دقیقه).

پس از تبدیل، `yt-dlp` فایل اصلی `m4a` را پاک می‌کند و فقط `mp3` باقی می‌ماند. الگوی خروجی تغییری نمی‌کند؛ پس فایل صوتی هم همان عنوان دقیق را می‌گیرد و پلی‌لیست هم همان پوشه شماره‌گذاری شده را.

۴.۳.۴ کوکی‌ها: وقتی یوتیوب از شما می‌خواهد وارد شوید

گاهی یوتیوب با پیام `Sign in to confirm you're not a bot` پاسخ می‌دهد. این اتفاق با `VPN` و شبکه‌های اشتراکی بیشتر رخ می‌دهد. **کوکی** داده کوچکی است که یک وبسایت در مرورگر شما ذخیره می‌کند تا ورود شما را به خاطر بسپارد. با `yt-dlp`، `cookiesfrombrowser` کوکی‌های یوتیوب یکی از مرورگرهای روی رایانه خودتان را می‌خواند و درست مانند همان مرورگر از ورود شما استفاده می‌کند.

نکته مهم

کوکی‌ها به اندازه گذرواژه خصوصی هستند. برنامه آن‌ها را فقط روی رایانه خودتان می‌خواند و هرگز جایی ذخیره یا ارسال نمی‌کند. فایل کوکی را هرگز با کسی به اشتراک نگذارید و هرگز در گیت قرار ندهید.

ایستگاه بررسی

بدون اجرای چیزی، نام فایلی را بنویسید که الگوی زیر

```
%(playlist_index)02d - %(title)s.%(ext)s
```

برای ویدیوی هفتم با عنوان Functions و پسوند mp4 می‌سازد. پاسخ: Functions.mp4 - 07.

فصل ۵

نوشتن گام به گام برنامه

هدف فصل

فایل `downloader.py` را تکه تکه بسازید و همه خطهای آن را بفهمید. در پایان این فصل برنامه کامل است.

یک فایل خالی به نام `downloader.py` در پوشه `code` بسازید و هر تکه زیر را به ترتیب در آن تایپ کنید. شماره‌های کنار کد، شماره خطها در فایل نهایی هستند؛ پس همیشه می‌بینید هر تکه کجا قرار می‌گیرد. میان تابعها دو خط خالی بگذارید؛ این سبک رایج پایتون است (PEP 8).

۱.۵ گام ۱: توصیف فایل

```
1 """Download one YouTube video or a whole playlist with yt-dlp.
2
3 Examples:
4     python downloader.py "VIDEO_OR_PLAYLIST_LINK"
5     python downloader.py "VIDEO_OR_PLAYLIST_LINK" -o ~/Videos
6     python downloader.py "VIDEO_OR_PLAYLIST_LINK" --audio
7     python downloader.py "VIDEO_OR_PLAYLIST_LINK" --browser firefox
8     python downloader.py          # asks for the link
9 """
```

رشته‌ای که در ابتدای فایل می‌آید، **داکاسترینگ ماژول** نام دارد. پایتون هنگام اجرا آن را نادیده می‌گیرد، اما به هر خواننده‌ای می‌گوید این فایل چه کاری انجام می‌دهد و چگونه به کار می‌رود.

۲.۵ گام ۲: وارد کردن ابزارها و تعیین پوشه پیش فرض

```
11 import argparse
12 import shutil
13 from pathlib import Path
14
15 from yt_dlp import YoutubeDL
16
17 # When no folder is given, files go to a "downloads" folder next to this script.
18 SCRIPT_FOLDER = Path(__file__).resolve().parent
19 DOWNLOAD_FOLDER = SCRIPT_FOLDER / "downloads"
```

• `argparse` خط فرمان را می‌خواند، `shutil` به دنبال `FFmpeg` می‌گردد و `Path` با پوشه‌ها کار می‌کند (فصل ۳).

• `YouTubeDL` کلاس دانلودر از `yt_dlp` است (فصل ۴).

• `SCRIPT_FOLDER` پوشه‌ای است که `downloader.py` در آن قرار دارد.

• `DOWNLOAD_FOLDER` پوشه `downloads` در داخل آن است. عملگر `/` دو بخش یک مسیر را به هم وصل می‌کند (فصل ۳). این پوشه با نخستین دانلود خودکار ساخته می‌شود.

نوشتن نام با حروف بزرگ، قراردادی در پایتون برای مقدارهایی است که هرگز تغییر نمی‌کنند (ثابت‌ها). به کار بردن پوشه اسکرپیت به جای پوشه فعلی ترمینال باعث می‌شود نتیجه همیشه در یک جای مشخص و قابل پیش‌بینی باشد؛ حتی اگر برنامه را از جای دیگری اجرا کنید، مثلاً با `python code/downloader.py`. دلیل ذخیره فایل‌ها در پوشه جداگانه `downloads` در فصل ۷ آمده است.

۳.۵ گام ۳: تشخیص پلی‌لیست

```
۲۲ def is_playlist(url):
۲۳     """Return True when the link contains a playlist id (list=...)."""
۲۴     return "list=" in url
```

هر پیوند پلی‌لیست یوتیوب عبارت `list=` و پس از آن شناسه پلی‌لیست را دارد:

<code>https://www.youtube.com/playlist?list=PLxxxxxxx</code>	-> playlist
<code>https://www.youtube.com/watch?v=abc123&list=PLxxxxxxx</code>	-> playlist
<code>https://www.youtube.com/watch?v=abc123</code>	-> one video
<code>https://youtu.be/abc123</code>	-> one video

عبارت `"list=" in url` خودش `True` یا `False` است؛ پس می‌توانیم آن را مستقیم برگردانیم. پیوند ویدیویی که از داخل یک پلی‌لیست باز شده باشد هم `list=` دارد و در این حالت کل پلی‌لیست دانلود می‌شود. برای دانلود فقط همان ویدیو، پیوند را بدون بخش `&list=...` کپی کنید.

۴.۵ گام ۴: انتخاب نام فایل‌ها

```
۲۷ def file_name_template(url):
۲۸     """Tell yt-dlp how to name each downloaded file."""
۲۹     if is_playlist(url):
۳۰         # Playlist title/01 - Video title.mp4
۳۱         return "%(playlist_title)s/%(playlist_index)02d - %(title)s.%(ext)s"
۳۲     # Video title.mp4
۳۳     return "%(title)s.%(ext)s"
```

این تابع یکی از دو الگوی خروجی فصل ۴ را برمی‌گرداند. برای پلی‌لیست، نویسه `/` پوشه‌ای به نام پلی‌لیست می‌سازد و شماره، ترتیب اصلی را حفظ می‌کند. برای یک ویدیو، فایل فقط به نام عنوانش ذخیره می‌شود. به `return` زود هنگام دقت کنید: اگر پیوند پلی‌لیست باشد، تابع در خط ۳۱ تمام می‌شود؛ پس به `else` نیازی نیست.

۵.۵ گام ۵: انتخاب کیفیت

```

۳۶ def video_quality():
۳۷     """Choose the best quality that this computer can save."""
۳۸     if shutil.which("ffmpeg"):
۳۹         # Best video + best audio, joined into one file by ffmpeg.
۴۰         return "bestvideo*+bestaudio/best"
۴۱     # Without ffmpeg, take the best file that already contains both.
۴۲     return "best"

```

shutil.which("ffmpeg") وقتی FFmpeg نصب باشد یک مسیر و وقتی نصب نباشد None برمی‌گرداند. با FFmpeg بهترین تصویر به‌علاوه بهترین صدا را درخواست می‌کنیم و بدون آن بهترین فایل تکی را؛ پس برنامه به‌جای توقف با خطا، باز هم کار می‌کند.

۶.۵ گام ۶: جمع‌کردن همه تنظیم‌ها

```

۴۵ def build_options(url, folder, browser=None, audio=False):
۴۶     """Collect all settings in the dictionary that yt-dlp expects."""
۴۷     options = {
۴۸         "format": video_quality(),
۴۹         # Prefer H264. video and AAC (m4a) audio: an mp4 that plays everywhere.
۵۰         "format_sort": ["vcodec:h264", "res", "acodec:m4a"],
۵۱         "merge_output_format": "mp4",
۵۲         "paths": {"home": str(folder)},
۵۳         "outtmpl": file_name_template(url),
۵۴         "ignoreerrors": True, # skip a private or deleted playlist video
۵۵         # Retry after a network drop (the Python default is 0 retries).
۵۶         "retries": 10,
۵۷     }
۵۸     if audio:
۵۹         # Only the sound: the best audio stream, converted to mp3 by ffmpeg.
۶۰         options["format"] = "bestaudio/best"
۶۱         if shutil.which("ffmpeg"):
۶۲             options["postprocessors"] = [
۶۳                 {"key": "FFmpegExtractAudio", "preferredcodec": "mp3",
۶۴                  "preferredquality": "192"},
۶۵             ]
۶۶     if browser:
۶۷         # Reuse the YouTube login of this browser when YouTube asks for it.
۶۸         options["cookiesfrombrowser"] = (browser,)
۶۹     return options

```

این تابع دیکشنری مورد انتظار YoutubeDL را می‌سازد. همه کلیدها در جدول‌های فصل ۴ توضیح داده شده‌اند.

- مقدار format و outtmpl از دو تابع کوچکی می‌آیند که همین حالا نوشتیم. تابع‌های کوچک با نام‌های روشن، خواندن تنظیم‌های اصلی را آسان می‌کنند.
- paths پوشه را به شکل رشته (str(folder)) دریافت می‌کند، چون yt-dlp همین را انتظار دارد.
- retries مهم است. **دستور** ترمینالی yt-dlp به‌طور پیش‌فرض ۱۰ بار دوباره تلاش می‌کند، اما مقدار پیش‌فرض آن در **پایتون** صفر است. بدون این خط، یک قطعی کوتاه شبکه دانلود تصویر را متوقف می‌کند (داستان کامل در فصل ۷ آمده است).
- خط‌های ۵۸ تا ۶۴ فقط وقتی اجرا می‌شوند که audio برابر True باشد. این خط‌ها قالب را با bestaudio/best (فقط صدا) **جایگزین** می‌کنند و اگر FFmpeg وجود داشته باشد، یک **پس‌پردازنده**

اضافه می‌کنند که نتیجه را به فایل MP3 با کیفیت ۱۹۲ کیلو بیت بر ثانیه تبدیل می‌کند. بدون FFmpeg فایل صوتی اصلی (معمولاً m4a) نگه داشته می‌شود که بیشتر پخش‌کننده‌ها آن را هم باز می‌کنند.

- `browser=None` و `audio=False` مقدارهای پیش‌فرض هستند: وقتی فراخواننده آن‌ها را ندهد، تنظیم کوکی اضافه نمی‌شود (خط ۶۶) و ویدیوی کامل دانلود می‌شود.

- `(browser,)` یک تاپل یک‌عضوی است؛ `yt-dlp` در اینجا تاپل می‌خواهد، چون می‌تواند نام پروفایل مرورگر را هم به‌عنوان عضو دوم بپذیرد.

ساختن دیکشنری در ابتدا و سپس تغییر چند کلید با بلوک‌های `if` یک الگوی رایج است: حالت عادی خوانا می‌ماند و هر حالت خاص یک بلوک کوچک و جداگانه است.

۷.۵ گام ۷: دانلود

```
۷۲ def download(url, folder=DOWNLOAD_FOLDER, browser=None, audio=False):
۷۳     """Download the link into folder. Return 0 when everything succeeded."""
۷۴     folder = Path(folder).expanduser().resolve()
۷۵     folder.mkdir(parents=True, exist_ok=True)
۷۶     print(f"Saving to: {folder}")
۷۷
۷۸     with YoutubeDL(build_options(url, folder, browser, audio)) as ydl:
۷۹         return ydl.download([url])
```

۱. خط ۷۴ هر چیزی را که کاربر نوشته است، به مسیر کامل تبدیل می‌کند: `expanduser` نویسه `~` را با پوشه خانه جایگزین می‌کند و `resolve` مسیر نسبی‌ای مانند `videos` را مطلق می‌کند.

۲. خط ۷۵ اگر پوشه هنوز وجود نداشته باشد، آن را می‌سازد.

۳. خط ۷۶ محل دقیق را چاپ می‌کند تا کاربر همیشه بداند کجا را نگاه کند.

۴. خط‌های ۷۸ و ۷۹ دالودر را با تنظیم‌های ما می‌سازند، پیوند را دانلود می‌کنند و نتیجه `yt-dlp` را برمی‌گردانند: `0` برای موفقیت کامل.

۸.۵ گام ۸: خواندن خط فرمان

```
۸۲ def main():
۸۳     parser = argparse.ArgumentParser(
۸۴         description="Download a YouTube video or a whole playlist."
۸۵     )
۸۶     parser.add_argument("url", nargs="?", help="video or playlist link")
۸۷     parser.add_argument(
۸۸         "-o", "--output", default=DOWNLOAD_FOLDER,
۸۹         help="folder for the files (default: downloads/ next to this script)",
۹۰     )
۹۱     parser.add_argument(
۹۲         "-a", "--audio", action="store_true",
۹۳         help="save only the sound, as an mp3 file",
۹۴     )
۹۵     parser.add_argument(
۹۶         "--browser",
۹۷         help="use this browser's YouTube login, e.g. firefox or chrome",
۹۸     )
```

```

۹۹     args = parser.parse_args()
۱۰۰
۱۰۱     url = args.url or input("Paste a YouTube video or playlist link: ").strip()
۱۰۲     url = url.replace("\\", "") # zsh may paste ? and = as \? and \=
۱۰۳     if not url:
۱۰۴         parser.error("a link is required")
۱۰۵
۱۰۶     try:
۱۰۷         result = download(url, args.output, args.browser, args.audio)
۱۰۸     except KeyboardInterrupt:
۱۰۹         print("\nStopped. Run the same command again to continue.")
۱۱۰         return
۱۱۱
۱۱۲     if result == 0:
۱۱۳         print("Done. All files were downloaded.")
۱۱۴     else:
۱۱۵         print("Finished, but some files could not be downloaded (see the errors
above).")

```

main کاربر را به تابع‌های بالا وصل می‌کند.

- url موقعیتی و اختیاری است ("?"=nargs).
- -o یا --output پوشه را انتخاب می‌کند و مقدار پیش‌فرضش DOWNLOAD_FOLDER است.
- -a یا --audio از action="store_true" استفاده می‌کند و مقداری نمی‌گیرد. نوشتن --audio مقدار args.audio را True می‌کند و ننوشتن آن، False (فصل ۳).
- --browser اختیاری است؛ اگر داده نشود، مقدارش None است.
- خط ۱۰۱: اگر پس از دستور پیوندی نوشته نشده باشد، args.url برابر None است؛ پس or تابع input را صدا می‌زند و پیوند را می‌پرسد.
- خط ۱۰۲ همه بک‌اسلش‌ها را حذف می‌کند. وقتی پیوندی را در zsh (پوسته پیش‌فرض ترمینال در مک و بسیاری از لینوکس‌ها) جای‌گذاری می‌کنید، ممکن است پیش از ? و = نویسه \ اضافه کند. پیوند واقعی هرگز بک‌اسلش ندارد؛ پس حذف آن همیشه بی‌خطر است (فصل ۷).
- خط‌های ۱۰۳ و ۱۰۴: اگر کاربر فقط Enter را بزند، parser.error پیامی روشن چاپ می‌کند و برنامه را متوقف می‌کند.
- خط‌های ۱۰۶ تا ۱۱۰ داندلود را داخل try اجرا می‌کنند. اگر کاربر Ctrl+C را بزند، پایتون خطای KeyboardInterrupt را ایجاد می‌کند؛ برنامه به‌جای یک گزارش خطای طولانی، یک جمله دوستانه چاپ می‌کند و return تابع main را تمام می‌کند.
- خط‌های ۱۱۲ تا ۱۱۵ بر اساس نتیجه، پیام پایانی را چاپ می‌کنند.

۹.۵ گام ۹: شروع برنامه

```

۱۱۸ if __name__ == "__main__":
۱۱۹     main()

```

این دو خط فقط وقتی main را صدا می‌زنند که فایل مستقیم اجرا شده باشد، نه وقتی فایل آزمون آن را وارد می‌کند (فصل ۳).

۱۰.۵ برنامه کامل

اکنون فایل شما باید دقیقاً مانند فهرست زیر باشد. اگر چیزی کار نکرد، آن‌ها را خطبه‌خط مقایسه کنید.

Listing :1.۵ The complete code in code/downloader.py

```
1 """Download one YouTube video or a whole playlist with yt-dlp.
2
3 Examples:
4     python downloader.py "VIDEO_OR_PLAYLIST_LINK"
5     python downloader.py "VIDEO_OR_PLAYLIST_LINK" -o ~/Videos
6     python downloader.py "VIDEO_OR_PLAYLIST_LINK" --audio
7     python downloader.py "VIDEO_OR_PLAYLIST_LINK" --browser firefox
8     python downloader.py          # asks for the link
9 """
10
11 import argparse
12 import shutil
13 from pathlib import Path
14
15 from yt_dlp import YoutubeDL
16
17 # When no folder is given, files go to a "downloads" folder next to this script.
18 SCRIPT_FOLDER = Path(__file__).resolve().parent
19 DOWNLOAD_FOLDER = SCRIPT_FOLDER / "downloads"
20
21
22 def is_playlist(url):
23     """Return True when the link contains a playlist id (list=...)."""
24     return "list=" in url
25
26
27 def file_name_template(url):
28     """Tell yt-dlp how to name each downloaded file."""
29     if is_playlist(url):
30         # Playlist title/01 - Video title.mp4
31         return "%(playlist_title)s/%(playlist_index)02d - %(title)s.%(ext)s"
32     # Video title.mp4
33     return "%(title)s.%(ext)s"
34
35
36 def video_quality():
37     """Choose the best quality that this computer can save."""
38     if shutil.which("ffmpeg"):
39         # Best video + best audio, joined into one file by ffmpeg.
40         return "bestvideo*+bestaudio/best"
41     # Without ffmpeg, take the best file that already contains both.
42     return "best"
43
44
45 def build_options(url, folder, browser=None, audio=False):
46     """Collect all settings in the dictionary that yt-dlp expects."""
47     options = {
48         "format": video_quality(),
49         # Prefer H264. video and AAC (m4a) audio: an mp4 that plays everywhere.
50         "format_sort": ["vcodec:h264", "res", "acodec:m4a"],
51         "merge_output_format": "mp4",
```

```

52     "paths": {"home": str(folder)},
53     "outtmpl": file_name_template(url),
54     "ignoreerrors": True, # skip a private or deleted playlist video
55     # Retry after a network drop (the Python default is 0 retries).
56     "retries": 10,
57 }
58 if audio:
59     # Only the sound: the best audio stream, converted to mp3 by ffmpeg.
60     options["format"] = "bestaudio/best"
61     if shutil.which("ffmpeg"):
62         options["postprocessors"] = [
63             {"key": "FFmpegExtractAudio", "preferredcodec": "mp3",
64              "preferredquality": "192"},
65         ]
66 if browser:
67     # Reuse the YouTube login of this browser when YouTube asks for it.
68     options["cookiesfrombrowser"] = (browser,)
69 return options
70
71 def download(url, folder=DOWNLOAD_FOLDER, browser=None, audio=False):
72     """Download the link into folder. Return 0 when everything succeeded."""
73     folder = Path(folder).expanduser().resolve()
74     folder.mkdir(parents=True, exist_ok=True)
75     print(f"Saving to: {folder}")
76
77     with YoutubeDL(build_options(url, folder, browser, audio)) as ydl:
78         return ydl.download([url])
79
80 def main():
81     parser = argparse.ArgumentParser(
82         description="Download a YouTube video or a whole playlist."
83     )
84     parser.add_argument("url", nargs="?", help="video or playlist link")
85     parser.add_argument(
86         "-o", "--output", default=DOWNLOAD_FOLDER,
87         help="folder for the files (default: downloads/ next to this script)",
88     )
89     parser.add_argument(
90         "-a", "--audio", action="store_true",
91         help="save only the sound, as an mp3 file",
92     )
93     parser.add_argument(
94         "--browser",
95         help="use this browser's YouTube login, e.g. firefox or chrome",
96     )
97     args = parser.parse_args()
98
99     url = args.url or input("Paste a YouTube video or playlist link: ").strip()
100     url = url.replace("\\", "") # zsh may paste ? and = as \? and \=
101     if not url:
102         parser.error("a link is required")
103
104     try:
105         result = download(url, args.output, args.browser, args.audio)
106     except KeyboardInterrupt:

```

```

109         print("\nStopped. Run the same command again to continue.")
110         return
111
112     if result == 0:
113         print("Done. All files were downloaded.")
114     else:
115         print("Finished, but some files could not be downloaded (see the errors
116         above).")
117
118 if __name__ == "__main__":
119     main()

```

ایستگاه بررسی

در پوشهٔ code دستور `python downloader.py --help` را اجرا کنید. باید خط کاربرد و چهار آرگومان `url`، `OUTPUT`، `-a`، و `--browser BROWSER` را ببینید. اگر پایتون خطای `SyntaxError` یا `IndentationError` نشان داد، خطی را که نام می‌برد با فهرست بالا مقایسه کنید.

فصل ۶

اجرا و آزمون برنامه

هدف فصل

یک ویدیو و یک پلی‌لیست را دانلود کنید، پوشه ذخیره را انتخاب کنید و آزمون‌های آفلاین را اجرا کنید.

مطمئن شوید محیط فعال است (فصل ۲) و ترمینال در پوشه code قرار دارد. در بعضی سیستم‌ها باید به جای python بنویسید python3.

۱.۶ نمایش صفحه راهنما

```
python downloader.py --help
```

نتیجه مورد انتظار

```
usage: downloader.py [-h] [-o OUTPUT] [-a] [--browser BROWSER] [url]

Download a YouTube video or a whole playlist.

positional arguments:
  url                  video or playlist link

options:
  -h, --help          show this help message and exit
  -o OUTPUT, --output OUTPUT
                      folder for the files (default: downloads/ next to
                      this
                      script)
  -a, --audio          save only the sound, as an mp3 file
  --browser BROWSER    use this browser's YouTube login, e.g. firefox or
                      chrome
```

۲.۶ دانلود یک ویدیو

از نخستین ویدیویی که در یوتیوب بارگذاری شد استفاده می‌کنیم، چون فقط ۱۹ ثانیه است:

```
python downloader.py "https://www.youtube.com/watch?v=jNQXAC9IVRw"
```

yt-dlp کاری را که انجام می‌دهد و یک خط پیشرفت زنده چاپ می‌کند. خط‌های مهم به این شکل هستند (مسیرها کوتاه شده‌اند):

نتیجه مورد انتظار

```
Saving to: /home/sara/prj-youtube-downloader/code/downloads
[youtube] Extracting URL: https://www.youtube.com/watch?v=jNQXAC9IVRw
[info] jNQXAC9IVRw: Downloading 1 format(s): 18
[download] Destination: .../code/downloads/Me at the zoo.mp4
[download] 100% of ...
Done. All files were downloaded.
```

اکنون فایل Me at the zoo.mp4 در پوشه code/downloads قرار دارد. این ویدیوی قدیمی نسخه HD ندارد؛ پس یک فایل آماده (قالب 18) کافی بود. برای یک ویدیوی HD دو شماره قالب، مانند 137+140، و یک خط اضافه ... Merging formats into ... [Merger] می‌بینید؛ این همان FFmpeg است که تصویر و صدای جدا را ترکیب می‌کند. شماره قالب‌ها ممکن است روی رایانه شما فرق کند؛ این طبیعی است.

۳.۶ دانلود یک پلی‌لیست کامل

یک پلی‌لیست یوتیوب را باز کنید، پیوند را از نوار نشانی کپی کنید و اجرا کنید:

```
python downloader.py "https://www.youtube.com/playlist?list=PLAYLIST_ID"
```

نتیجه مورد انتظار

پوشه‌ای تازه به نام پلی‌لیست در code/downloads ساخته می‌شود. ویدیوهای داخل آن به ترتیب پلی‌لیست شماره‌گذاری شده‌اند و عنوان دقیق خود را نگه می‌دارند؛ برای نمونه:

```
Get more creative with AI/
|-- 01 - Vibe coding in Google AI Studio is changing the game.mp4
|-- 02 - 3 Incredible #NanoBanana Community Builds.mp4
`-- ...
```

دانلود یک پلی‌لیست بزرگ ممکن است زمان زیادی بگیرد. هر لحظه می‌توانید با Ctrl+C آن را متوقف کنید. اگر بعداً همان دستور را دوباره اجرا کنید، yt-dlp ویدیوهای کامل‌شده را رد می‌کند و بقیه را ادامه می‌دهد.

۴.۶ ذخیره فقط صدا

گزینه -a یا --audio را اضافه کنید. این گزینه هم برای یک ویدیو و هم برای پلی‌لیست کار می‌کند:

```
python downloader.py "https://youtu.be/huwi-cjPPXU" --audio
```

نتیجه مورد انتظار

```
[info] huwi-cjPPXU: Downloading 1 format(s): 140
[download] 100% of 00.14MiB in 00:00:04 at 98.2MiB/s
[ExtractAudio] Destination: ../code/downloads/Introduce Yourself
in Slow German (for Absolute Beginners).mp3
Deleting original file ... .m4a (pass -k to keep)
Done. All files were downloaded.
```

قالب 140 صدای AAC یوتیوب است. خط ExtractAudio همان FFmpeg است که آن را به MP3 تبدیل می‌کند؛ سپس فایل موقت m4a، پاک می‌شود. این درس ۱۵ دقیقه‌ای به یک فایل MP3 ۲۱ مگابایتی تبدیل می‌شود و به جای ۱۴۴ مگابایت ویدیوی Full HD فقط ۱۴ مگابایت دانلود لازم دارد.

۵.۶ انتخاب پوشه ذخیره

گزینه ۰- و یک پوشه را اضافه کنید. اگر پوشه وجود نداشته باشد، ساخته می‌شود:

```
python downloader.py "LINK" -o ~/Videos # macOS / Linux
python downloader.py "LINK" -o "D:\My Videos" # Windows
python downloader.py "LINK" -o downloads # a folder here
```

خط اول خروجی، یعنی Saving to: ... همیشه مسیر کامل را نشان می‌دهد. اگر نام پوشه فاصله دارد، آن را داخل گیومه بنویسید.

۶.۶ روش‌های دیگر اجرا

```
python downloader.py # asks for the link
python downloader.py "LINK" --browser firefox # use Firefox's YouTube login
python downloader.py "LINK" --browser chrome -o ~/Videos
```

بدون پیوند، برنامه پیام Paste a YouTube video or playlist link: را نشان می‌دهد. پیوند را جای‌گذاری کنید (در ترمینال لینوکس معمولاً با Ctrl+Shift+V و در ویندوز با کلیک راست) و Enter را بزنید. گزینه --browser - - را فقط وقتی به کار ببرید که یوتیوب از شما بخواهد وارد شوید (فصل ۸). نام‌های پشتیبانی‌شده شامل safari و opera, brave, edge, chromium, chrome, firefox هستند.

۷.۶ توقف و ادامه در زمانی دیگر

هر لحظه می‌توانید Ctrl+C را بزنید. برنامه پاسخ می‌دهد:

نتیجه مورد انتظار

```
Stopped. Run the same command again to continue.
```

بخش نیمه‌کاره به شکل یک فایل part. در پوشه می‌ماند. وقتی همان دستور را دوباره اجرا کنید، yt-dlp پیام Resuming download at byte ... را چاپ می‌کند و به جای شروع دوباره، از همان‌جا ادامه می‌دهد.

۸.۶ آزمون‌های آفلاین

آزمون‌ها برنامه‌های کوچکی هستند که کدهای دیگر را خودکار بررسی می‌کنند. آزمون‌های ما در کسری از ثانیه اجرا می‌شوند و به اینترنت نیاز ندارند، چون فقط بررسی می‌کنند برنامه دانلود را چگونه آماده می‌کند. فایل `test_downloader.py` را کنار `downloader.py` بسازید:

Listing ۱.۶ The complete code in `code/test_downloader.py`

```
1 """Offline checks for downloader.py. Run with: python -m unittest -v"""
2
3 import unittest
4 from pathlib import Path
5 from unittest.mock import patch
6
7 import downloader
8
9 VIDEO = "https://www.youtube.com/watch?v=abc123"
10 PLAYLIST = "https://www.youtube.com/playlist?list=PL123"
11
12
13 class DownloaderTests(unittest.TestCase):
14     def test_detects_playlist_and_single_video(self):
15         self.assertTrue(downloader.is_playlist(PLAYLIST))
16         self.assertFalse(downloader.is_playlist(VIDEO))
17
18     def test_single_video_is_named_by_title(self):
19         self.assertEqual(downloader.file_name_template(VIDEO), "%(title)s.%(ext)s"
20 )
21
22     def test_playlist_gets_its_own_folder(self):
23         name = downloader.file_name_template(PLAYLIST)
24         self.assertTrue(name.startswith("%(playlist_title)s/"))
25
26     def test_default_folder_is_downloads_next_to_script(self):
27         script_folder = Path(downloader.__file__).resolve().parent
28         self.assertEqual(downloader.DOWNLOAD_FOLDER, script_folder / "downloads")
29
30     def test_options_use_given_folder_and_browser(self):
31         options = downloader.build_options(VIDEO, "/tmp/videos", "firefox")
32         self.assertEqual(options["paths"], {"home": "/tmp/videos"})
33         self.assertEqual(options["cookiesfrombrowser"], ("firefox",))
34
35     def test_retries_after_network_drop(self):
36         self.assertEqual(downloader.build_options(VIDEO, "/tmp/videos")["retries"
37 ], 10)
38
39     def test_audio_only_becomes_mp3(self):
40         with patch("shutil.which", return_value="/usr/bin/ffmpeg"):
41             options = downloader.build_options(VIDEO, "/tmp/videos", audio=True)
42             self.assertEqual(options["format"], "bestaudio/best")
43             self.assertEqual(options["postprocessors"][0]["preferredcodec"], "mp3")
44
45     def test_audio_only_without_ffmpeg_keeps_original_file(self):
46         with patch("shutil.which", return_value=None):
47             options = downloader.build_options(VIDEO, "/tmp/videos", audio=True)
48             self.assertEqual(options["format"], "bestaudio/best")
49             self.assertNotIn("postprocessors", options)
```

```

۴۸
۴۹     def test_quality_without_ffmpeg(self):
۵۰         with patch("shutil.which", return_value=None):
۵۱             self.assertEqual(downloader.video_quality(), "best")
۵۲
۵۳
۵۴ if __name__ == "__main__":
۵۵     unittest.main()

```

• unittest ماژول آزمون داخلی پایتون است. هر متدی داخل یک کلاس unittest.TestCase که نامش با test_ شروع شود، یک آزمون است.

• assertTrue, assertFalse و assertEquals یک نتیجه را بررسی می‌کنند؛ اگر بررسی نادرست باشد، آزمون شکست می‌خورد و هر دو مقدار را نشان می‌دهد.

• patch یک تابع را موقتاً جایگزین می‌کند. در آزمون آخر، shutil.which وانمود می‌کند وجود ندارد؛ پس می‌توانیم آن حالت را حتی روی رایانه‌ای که FFmpeg دارد هم آزمون کنیم. دو آزمون صدا همین ترفند را در هر دو جهت به کار می‌برند: وجود FFmpeg (تبدیل به MP3 اضافه می‌شود) و نبود آن (فایل صوتی اصلی نگه داشته می‌شود).

همهٔ آزمون‌ها را اجرا کنید:

```
python -m unittest -v
```

نتیجهٔ مورد انتظار

```

test_audio_only_becomes_mp3 ... ok
test_audio_only_without_ffmpeg_keeps_original_file ... ok
test_default_folder_is_downloads_next_to_script ... ok
test_detects_playlist_and_single_video ... ok
test_options_use_given_folder_and_browser ... ok
test_playlist_gets_its_own_folder ... ok
test_quality_without_ffmpeg ... ok
test_retries_after_network_drop ... ok
test_single_video_is_named_by_title ... ok
-----
Ran 9 tests in 0.010s

OK

```

هر بار که downloader.py را تغییر دادید، آزمون‌ها را دوباره اجرا کنید. اگر رفتاری را عمداً تغییر دادید، آزمون متناظر را هم به‌روز کنید.

ایستگاه بررسی

یک ویدیو را در پوشهٔ code/downloads دانلود کرده‌اید، یک فایل صوتی را با --audio ذخیره کرده‌اید، یک پلی‌لیست (یا بخشی از آن) را در پوشهٔ خودش دانلود کرده‌اید، یک بار از -o استفاده کرده‌اید و پیام OK را از آزمون‌ها دیده‌اید.

فصل ۷

درس‌هایی از استفاده واقعی

هدف فصل

ببینید نسخه نخست برنامه پس از استفاده روی ویدیوهای واقعی چگونه بهتر شد: چه چیزی خراب شد، علت آن چگونه پیدا شد، چرا هر اصلاح درست کار می‌کند و چگونه آزمون شد. این‌ها عادت‌هایی هستند که یک اسکریپت‌کارکن را به ابزاری قابل‌اعتماد تبدیل می‌کنند.

نسخه نخست downloader.py همه آزمون‌هایش را با موفقیت گذراند و روی ویدیوهای کوتاه کار کرد. سپس استفاده واقعی مشکل‌هایی را نشان داد که هیچ آزمونی آن‌ها را پوشش نمی‌داد. این فصل همان داستان را تعریف می‌کند، چون پیدا کردن یک مشکل از هر کد آماده‌ای آموزنده‌تر است. همه تغییرهایی که اینجا توضیح داده می‌شوند، از قبل در برنامه فصل ۵ وجود دارند.

۱.۷ روشی ساده برای هر باگ

۱. نشانه را دقیق توصیف کنید. چه انتظاری داشتید و چه اتفاقی افتاد؟
۲. کل خروجی را بخوانید، به‌ویژه نخستین هشدار یا خطا را.
۳. شواهد را بررسی کنید: کدام فایل‌ها وجود دارند، چه حجمی دارند و نامشان چه می‌گوید.
۴. درباره علت حدس بزنید و سپس آن را بررسی کنید؛ در مستندات یا کد منبع کتابخانه. فقط بر اساس حدس، کد را تغییر ندهید.
۵. علت را اصلاح کنید، نه نشانه را؛ با کوچک‌ترین تغییر روشن.
۶. یک آزمون اضافه کنید که این مشکل را پیدا می‌کند و حالت واقعی را دوباره امتحان کنید.

۲.۷ درس ۱: قطع شبکه نیمی از ویدیو را جا گذاشت

۱.۲.۷ نشانه

یک کاربر یک درس ۱۵ دقیقه‌ای با کیفیت Full HD را با این دستور دانلود کرد:

```
python downloader.py "https://youtu.be/huwi-cjPPXU?si=Huhcwi9PkWR87c-u" -o ~/Videos
```

خروجی یک خطا داشت و برنامه با پیام «برخی فایل‌ها دانلود نشدند» تمام شد:

```
[info] huwi-cjPPXU: Downloading 1 format(s): 137+140
[download] Destination: ...Beginners).f137.mp4
[download] Got error: 9849475 bytes read, 318544 more expected:
[download] Destination: ...Beginners).f140.m4a
[download] 100% of 00.14MiB in 00:01:58 at 43.121KiB/s
Finished, but some files could not be downloaded (see the errors above).
```

در پوشه یک فایل نیمه‌کاره `f137.mp4.part` بود که تصویر داشت اما **صدانداشت** و یک فایل `f140.m4a` که **فقط صدا** داشت. به نظر می‌رسید برنامه ویدیو را دو بار و در دو قالب اشتباه دانلود کرده است.

۲.۲.۷ خواندن شواهد

- `137+140` یعنی دو قالب: `137` جریان تصویر `1080p` و `140` جریان صدای AAC است (فصل ۴). دانلود دو فایل **طبیعی** است؛ FFmpeg آن‌ها را در پایان کار ترکیب می‌کند. پس پرسش واقعی این است: چرا ترکیبی انجام نشد؟
- `Got error: ... more expected` یعنی اتصال پیش از کامل شدن بخش تصویر بسته شده است. سرعت (`121 KiB/s`) هم نشان می‌دهد شبکه کند و ناپایدار بوده است.
- هیچ خط (`1/10`) Retrying دیده نمی‌شود. دانلود در نخستین خطا رها شده و بدون بخش کامل تصویر، FFmpeg چیزی برای ترکیب نداشته است.

۳.۲.۷ پیدا کردن علت

همه می‌دانند **دستور** ترمینالی `yt-dlp` ۱۰ بار دوباره تلاش می‌کند. پس چرا اینجا نه؟ پاسخ در کد منبع کتابخانه است که `pip` آن را در پوشه `site-packages/yt_dlp` محیط نصب کرده است:

```
# options.py -- only used by the yt-dlp terminal command
'--retries', dest='retries', default=10, ...

# downloader/http.py -- used by everyone, including our program
for retry in RetryManager(self.params.get('retries'), self.report_retry):
```

مقدار پیش‌فرض ۱۰ متعلق به **تجزیه‌گر خط فرمان** است. وقتی YouTubeDL را از پایتون به کار می‌بریم، `params.get('retries')` چیزی پیدا نمی‌کند و `None` برمی‌گرداند که یعنی **صفر** بار تلاش دوباره.

نکته مهم

کتابخانه‌ای که از پایتون به کار می‌رود، همیشه مانند همان ابزار در ترمینال رفتار نمی‌کند. مقدارهای پیش‌فرض اغلب در لایه خط فرمان تعریف می‌شوند. وقتی تنظیمی مهم است، به جای تکیه بر مقدار پیش‌فرض، آن را صریح در تنظیم‌های خود بنویسید.

۴.۲.۷ اصلاح و آزمون

یک خط در `build_options` (فصل ۵، گام ۶):

```
"retries": 10,
```

یک آزمون تازه از آن محافظت می‌کند تا کسی آن را اشتباهی حذف نکند:

```
def test_retries_after_network_drop(self):
    self.assertEqual(downloader.build_options(VIDEO, "/tmp/videos")["retries"],
                      10)
```

سپس حالت واقعی دوباره امتحان شد. همان دستور فایل نیمه‌کاره را ادامه داد:

```
[download] Resuming download at byte 49650475
```

سپس صدا را دانلود کرد و FFmpeg آن‌ها را ترکیب کرد. بررسی با ffprobe (که همراه FFmpeg نصب می‌شود) نتیجه را ثابت کرد:

```
stream|codec_name=h264|codec_type=video|width=1920|height=1080|duration64.906=
stream|codec_name=aac|codec_type=audio|duration69.906=
```

هر دو جریان وجود دارند و هر دو کامل، یعنی ۱۵ دقیقه و ۶ ثانیه، هستند.

۳.۷ درس ۲: ترمینال پیوند را تغییر داد

۱.۳.۷ نشانه

دوباره به پیوند دستور بالا نگاه کنید: `.\?si\=`. کاربر یک پیوند معمولی را جای‌گذاری کرده بود. این بک‌اسلش‌ها را `zsh`، پوسته پیش‌فرض مک و بسیاری از لینوکس‌ها، اضافه کرده بود. هنگام جای‌گذاری یک نشانی، قابلیت از `zsh` به نام `url-quote-magic` از نویسه‌های ویژه‌ای مانند `?` و `=` با یک بک‌اسلش محافظت می‌کند؛ کاری که بدون گیومه درست است. اما داخل گیومه، بک‌اسلش‌ها بخشی از متن می‌شوند و پایتون یک پیوند خراب دریافت می‌کند.

۲.۳.۷ چرا مهم است

این ویدیو از روی شانس باز هم کار کرد، چون `yt-dlp` شناسه ویدیو را پیش از بخش خراب پیدا کرد. اما برای پیوند پلی‌لیست بی‌صدا شکست می‌خورد: `list=\` شامل `list=` نیست؛ پس `is_playlist` مقدار `False` برمی‌گرداند و کل پلی‌لیست بدون پوشه و شماره ذخیره می‌شد.

۳.۳.۷ اصلاح

نشانی معتبر هرگز بک‌اسلش ندارد؛ پس همیشه حذف آن‌ها درست در لحظه خواندن پیوند بی‌خطر است (فصل ۵، گام ۸):

```
url = url.replace("\\", "") # zsh may paste ? and = as \? and \=
```

درس این بخش: **ورودی کاربر را همان جایی که وارد برنامه می‌شود تمیز کنید.** آن‌گاه همه تابع‌های بعدی می‌توانند به آن مقدار اعتماد کنند.

۴.۷ درس ۳: دانلودها را از کد و از گیت جدا نگه دارید

در نسخه نخست، فایل‌ها بدون `-o` درست کنار `downloader.py` ذخیره می‌شدند. پس از چند دانلود، پوشه `code` پر از ویدیوهای حجیم شده بود که با فایل‌های کد قاطی بودند و یک `git add` بی‌دقت می‌توانست بخواهد گیگابایت‌ها داده را در گیت‌هاب بارگذاری کند.

اکنون پیش‌فرض یک پوشه جداگانه است:

```
SCRIPT_FOLDER = Path(__file__).resolve().parent
DOWNLOAD_FOLDER = SCRIPT_FOLDER / "downloads"
```

و فایل `.gitignore` خط `code/downloads/` را دارد. می‌توانید از گیت بپرسید آیا فایل نادیده گرفته می‌شود و با کدام قانون:

```
git check-ignore -v "code/downloads/Me at the zoo.mp4"
```

```
.gitignore:222:code/downloads/ code/downloads/Me at the zoo.mp4
```

اکنون دو محافظت مستقل وجود دارد: قانون پوشه و قانون‌های قدیمی‌تر برای پسوندهای رسانه‌ای مانند `*.mp4`. جدا کردن فایل‌های **برنامه** از فایل‌های **داده** عادت خوبی در هر پروژه‌ای است.

۵.۷ درس ۴: یک قابلیت تازه، فقط صدا

زبان‌آموزان، شنوندگان پادکست و دوستداران موسیقی اغلب فقط به صدا نیاز دارند. دانلود ویدیوی Full HD برای این کار وقت و فضا را هدر می‌دهد: درس بالا به شکل ویدیو ۱۴۴ مگابایت است، اما صدایش فقط ۱۴ مگابایت دانلود لازم دارد. طراحی، برنامه را کوچک نگه می‌دارد:

- یک کلید در خط فرمان: `-a` یا `-audio` با `action="store_true"` (فصل ۳).
 - یک پارامتر تازه، `audio=False`، که از `main` از طریق `download` به `build_options` می‌رسد. مقدار پیش‌فرض آن رفتار قبلی را حفظ می‌کند؛ پس دستورهای قبلی هیچ تغییری نمی‌کنند.
 - یک بلوک `if audio`: که فقط چیزهای متفاوت را تغییر می‌دهد. قالب به `bestaudio/best` تبدیل می‌شود و یک پس‌پردازنده `FFmpeg` نتیجه را به `MP3` تبدیل می‌کند (فصل ۴).
 - اگر `FFmpeg` نباشد، پس‌پردازنده اضافه **نمی‌شود**، چون شکست می‌خورد. به جای آن فایل صوتی اصلی نگه داشته می‌شود. این همان ایده «با امکانات همین رایانه کار کن» در `video_quality` است.
- دو آزمون هر دو حالت را پوشش می‌دهند. آن‌ها با `patch` و `انمود` می‌کنند `FFmpeg` وجود دارد یا ندارد؛ پس روی هر رایانه‌ای نتیجه یکسانی می‌دهند.

۶.۷ درس ۵: توقف مؤدبانه

زدن `Ctrl+C` در میانهٔ دانلود، پیش‌تر یک گزارش خطای طولانی چاپ می‌کرد که شبیه خرابی برنامه بود. اکنون `main` استثنای `KeyboardInterrupt` را می‌گیرد (فصل ۳) و یک جمله روشن چاپ می‌کند:

```
Stopped. Run the same command again to continue.
```

این جمله به دلیل روش کار `yt-dlp` درست است: دانلود ناتمام به شکل فایل `part`. نگه داشته می‌شود و اجرای بعدی آن را ادامه می‌دهد. پیام خوب به کاربر می‌گوید **چه اتفاقی افتاد و گام بعدی چیست**.

۷.۷ خلاصهٔ تغییرها

مشکل یا نیاز	تغییر و روش آزمون آن
قطع شبکه دانلود تصویر را متوقف کرد	10: "retries"؛ یک آزمون واحد و تکرار دانلود واقعی
zsh به پیوندها بک اسلش اضافه کرد	(...) url.replace در main؛ آزمون با یک پیوند واقعی تغییر یافته در zsh
ویدیوها با کد قاطی می شدند	DOWNLOAD_FOLDER و code/downloads/ در .gitignore؛ یک آزمون واحد و git check-ignore
فقط صدا	audio -- و پس پردازنده MP3؛ دو آزمون واحد و دانلود واقعی یک ویدیو و یک پلی لیست
Ctrl+C شبیه خرابی بود	KeyboardInterrupt try/except؛ آزمون با متوقف کردن یک دانلود واقعی

ایستگاه بررسی

می توانید توضیح دهید چرا دانلود Full HD پیش از فایل نهایی mp4. دو فایل می سازد، چرا مقدار پیش فرض retries در پایتون با دستور ترمینال فرق دارد و چرا برنامه بک اسلش ها را از پیوند حذف می کند.

فصل ۸

رفع اشکال و گام‌های بعدی

هدف فصل

مشکل‌های رایج را سریع برطرف کنید و یک بهبود کوچک و کاربردی را برای ساخت در گام بعد انتخاب کنید.

۱.۸ مشکل‌های رایج

پیام یا نشانه	راه حل
ModuleNotFoundError	محیط فعال نیست یا بسته‌ها نصب نشده‌اند. محیط را فعال کنید و دستور python -m pip install -r requirements.txt اجرا کنید.
Sign in to confirm you're not a bot	یوتیوب را در یک مرورگر باز کنید و وارد حساب خود شوید. مرورگر را ببندید و دستور را دوباره با --browser firefox (یا نام مرورگر خودتان) اجرا کنید. خاموش کردن VPN یا تغییر سرور آن هم می‌تواند کمک کند.
No supported JavaScript runtime	deno نصب نیست. محیط را فعال کنید یا python -m pip install deno را اجرا کنید.
Got error: ... bytes read, ... more expected و در پوشه یک فایل f137.mp4.part و یک فایل f140.m4a مانده است	هنگام دانلود بخش تصویر، شبکه قطع شده است. یوتیوب تصویر و صدا را در دو بخش جدا می‌فرستد و فقط وقتی هر دو کامل باشند آن‌ها را ترکیب می‌کند. برنامه خودش ۱۰ بار دوباره تلاش می‌کند؛ اگر باز هم شکست خورد، همان دستور را دوباره اجرا کنید. yt-dlp بخش نیمه‌کاره را ادامه می‌دهد، هر دو را ترکیب می‌کند و فایل‌های موقت را پاک می‌کند.
Could not copy Chrome cookie database	کروم باز است و کوکی‌هایش را قفل کرده است. همه پنجره‌های کروم را ببندید و دوباره امتحان کنید یا مرورگر دیگری را به کار ببرید.
فایل صدا ندارد یا فقط کیفیت پایین ذخیره می‌شود	FFmpeg نصب نیست یا پیدا نمی‌شود. آن را نصب کنید و در یک ترمینال تازه ffmpeg -version را بررسی کنید.
قبلاً کار می‌کرد، اما حالا با HTTP Error 403 یا خطای مشابه شکست می‌خورد	یوتیوب چیزی را تغییر داده است. کتابخانه را به‌روز کنید: python -m pip install -U "yt-dlp[default]"
Private یا Video unavailable video	این ویدیو بدون اجازه قابل تماشا نیست. ویدیوهای دیگر پلی‌لیست همچنان دانلود می‌شوند و برنامه با پیام «برخی فایل‌ها دانلود نشدند» تمام می‌شود.
zsh: no matches found یا پیوند در نویسه & بریده می‌شود	پیوند را داخل گیومه بنویسید.
به‌جای یک ویدیو، کل پلی‌لیست شروع می‌شود	پیوند بخش &list= دارد. آن بخش را حذف کنید و دوباره اجرا کنید.

وقتی کمک می‌خواهید (مثلاً در یک Issue در گیت‌هاب)، سیستم‌عامل، نسخه پایتون، دستور دقیق و پیام کامل خطا را بنویسید. پیش از آن، هر اطلاعات خصوصی را حذف کنید.

۲.۸ تمیز نگه داشتن پروژه در گیت

فایل gitignore. پروژه به گیت می‌گوید کدام فایل‌ها را هرگز ذخیره نکند. این فایل کل پوشه code/downloads/، فایل‌های رسانه‌ای در هر جای پروژه (مانند *.mp4، *.mp3 و *.webm)، فایل‌های نیمه‌کاره *.part و پوشه venv. را فهرست کرده است. پیش از هر کامیت دستور git status را اجرا کنید: هیچ فایل ویدیویی نباید در فهرست باشد.

۳.۸ ایده‌هایی برای تمرین

هر ایده فقط چند خط کد لازم دارد و یک مهارت تازه را تمرین می‌دهد. برای هر تغییر یک آزمون هم اضافه کنید.

۱. **انتخاب قالب صدا.** گزینه `--audio-format` را با `choices=["mp3", "m4a", "opus"]` اضافه کنید و مقدار آن را به جای "mp3" ثابت، در `preferredcodec` به کار ببرید:

```
parser.add_argument("--audio-format", choices=["mp3", "m4a", "opus"],
                    default="mp3", help="file type for --audio")
```

۲. **بیشینه کیفیت.** برای صرفه‌جویی در فضای دیسک، گزینه `--max-height` را با مقداری مانند 720 اضافه کنید:

```
options["format"] = "bestvideo*[height<=720]+bestaudio/best[height<=720]"
```

۳. **چند پیوند با هم.** `url` را به `nargs="*"` تغییر دهید و کل لیست را به `yd1.download` بدهید.

۴. **زیرنویس.** زیرنویس انگلیسی و فارسی را کنار هر ویدیو ذخیره کنید:

```
options["writesubtitles"] = True
options["subtitleslangs"] = ["en", "fa"]
```

۵. **خواندن پیوندها از فایل.** از فایل `links.txt` در هر خط یک پیوند بخوانید:

```
links = Path("links.txt").read_text().splitlines()
```

همه گزینه‌های موجود در **راهنمای yt-dlp** توضیح داده شده‌اند. گزینه‌ای که در خط فرمان نوشته می‌شود، مانند `--write-sub`، در پایتون به کلیدی در دیکشنری تبدیل می‌شود، مانند `writesubtitles`.

ایستگاه بررسی

می‌توانید علت یک دانلود ناموفق را از پیامش پیدا کنید، `yt-dlp` را به‌روز کنید و خودتان گزینه تازه‌ای به دیکشنری تنظیم‌ها اضافه کنید.

تبریک! شما یک ابزار کامل و آزمون‌شده پایتون را از یک پوشه خالی ساختید.

درباره نویسنده

ایده و محتوای این آموزش و پروژه را Alireza Khajehvandi ساخته است تا تازه‌کارها در زمانی کوتاه ابزاری کاربردی بسازند و هر خطی را که اجرا می‌کنند، بفهمند.

برای ادامه یادگیری و ارتباط:

-  LinkedIn
-  YouTube کانال
-  کانال تلگرام
-  Instagram
-  GitHub
-  وبسایت شخصی و نمونه‌کارها